Task 2 - Design a Kotlin Class and Data Model for an App Feature

1. Feature Choice: DSA Problem Tracker

Description:

This feature allows students to track their daily DSA practice by listing problems with their difficulty, status (Solved or Attempted), and a direct link to the problem.

It helps learners stay consistent by providing a clear overview of their progress across different data structures.

2. Modeling the Data

Kotlin Code:

// Data Class: Represents the immutable state of a single problem data class DSAProblem(val id: Int, val title: String, val difficulty: String, // Easy, Medium, Hard var isCompleted: Boolean = false, val url: String)

// Interface: Defines behavior for managing problems interface ProblemManager {

```
fun toggleStatus(problemId: Int)

fun getProblemsByDifficulty(level: String): List<DSAProblem>
```

}

// Regular Class: Implements behavior and manages state class TrackerViewModel : ProblemManager {

```
private val _problems = mutableListOf<DSAProblem>()

override fun toggleStatus(problemId: Int) {

    val problem = _problems.find { it.id == problemId }

    problem?.let { it.isCompleted = !it.isCompleted }

}

override fun getProblemsByDifficulty(level: String): List<DSAProblem> {

    return _problems.filter { it.difficulty == level }

}
```

}

Rationale:

I chose a data class for DSAProblem because its main purpose is storing data. Kotlin automatically generates methods like copy() and toString().

I used an interface for ProblemManager to separate behavior definition from implementation.

The regular class TrackerViewModel stores the problem list and implements the logic.

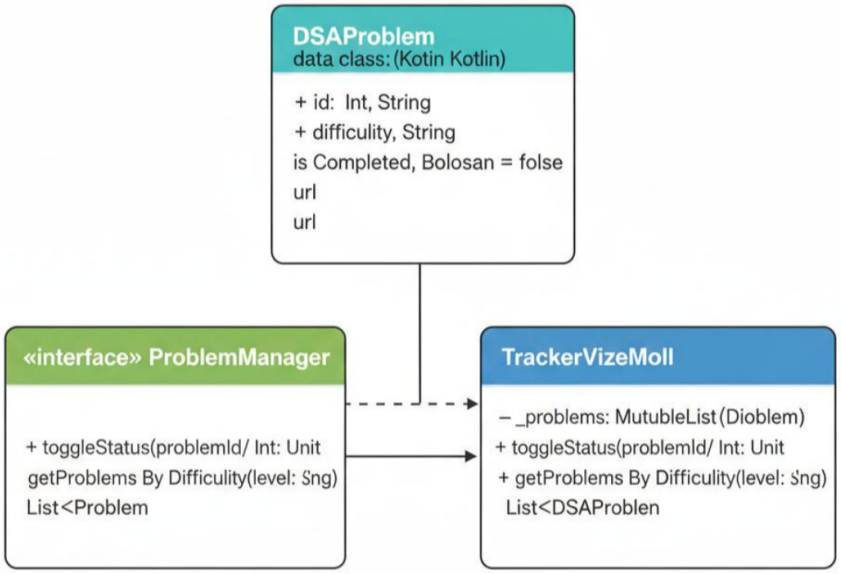
3. Class Relationship Diagram

Kotlin Features Used:

Primary Constructor: Used in DSAProblem for concise property declaration.

Default Parameter: isCompleted defaults to false.

Interface Implementation: TrackerViewModel uses ":" syntax to implement ProblemManager behavior.



4. Mapping to Compose UI

This model connects to Compose using LazyColumn to display the list of DSAProblem objects efficiently.

When toggleStatus() is called, isCompleted changes.

If the list is wrapped inside MutableState, recomposition will occur automatically and update the UI with a completed checkmark.

5. Reflection

The main challenge was deciding which class should own the logic and which should only store data.

Using object-oriented design ensures the UI only calls functions like toggleStatus() without knowing how data is stored or filtered.

This makes the application easier to maintain and scale in the future.